

EDGE INTELLIGENCE

ASSIGNMENT NO: 2

DONE BY
DHAYANIDHI R S
[25MML0025]

Task-1:

Key Take aways:

1. Analysing an image dataset-MNIST and applying basic preprocessing operations,
2. Applied Artificial neural network on the following dataset.
3. Saving the model using pickle model.

```
[1]
✓ 14s import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

[2]
✓ 1s # Load the dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

# Split a portion of training data for validation (e.g., 10,000 images)
val_images = train_images[-10000:]
val_labels = train_labels[-10000:]
train_images = train_images[:-10000]
train_labels = train_labels[:-10000]

print(f"Training samples: {len(train_images)}, Validation samples: {len(val_images)}, Test samples: {len(test_images)}")

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 1s 0us/step
Training samples: 50000, Validation samples: 10000, Test samples: 10000

[4]
✓ 0s # Convert to float32 FIRST, then divide and reshape
train_images = train_images.astype('float32') / 255.0
val_images = val_images.astype('float32') / 255.0
test_images = test_images.astype('float32') / 255.0

# Reshape to (samples, 784)
train_images = train_images.reshape((-1, 784))
val_images = val_images.reshape((-1, 784))
test_images = test_images.reshape((-1, 784))

[5]
✓ 0s model = models.Sequential([
# Input + Hidden Layer 1
layers.Dense(64, activation='relu', input_shape=(784,)),
layers.Dropout(0.2),

# Hidden Layer 2
layers.Dense(32, activation='relu'),
layers.Dropout(0.2),

# Output Layer
layers.Dense(10, activation='softmax')
])

model.summary()

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	50,240
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 10)	330

Total params: 52,650 (205.66 KB)
Trainable params: 52,650 (205.66 KB)
Non-trainable params: 0 (0.00 B)

```
[6]
✓ 0s model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

```
[7]
✓ 31s
from tensorflow.keras.callbacks import EarlyStopping

# Stop training if validation loss doesn't improve for 3 epochs
early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

history = model.fit(train_images, train_labels,
                    epochs=20,
                    batch_size=128,
                    validation_data=(val_images, val_labels),
                    callbacks=[early_stop])
```

```
Epoch 1/20
391/391 — 3s 5ms/step - accuracy: 0.6392 - loss: 1.1280 - val_accuracy: 0.9337 - val_loss: 0.2387
Epoch 2/20
391/391 — 2s 4ms/step - accuracy: 0.8933 - loss: 0.3648 - val_accuracy: 0.9499 - val_loss: 0.1769
Epoch 3/20
391/391 — 2s 4ms/step - accuracy: 0.9185 - loss: 0.2811 - val_accuracy: 0.9576 - val_loss: 0.1512
Epoch 4/20
391/391 — 3s 7ms/step - accuracy: 0.9333 - loss: 0.2312 - val_accuracy: 0.9598 - val_loss: 0.1382
Epoch 5/20
391/391 — 2s 4ms/step - accuracy: 0.9399 - loss: 0.2072 - val_accuracy: 0.9660 - val_loss: 0.1210
Epoch 6/20
391/391 — 2s 4ms/step - accuracy: 0.9457 - loss: 0.1846 - val_accuracy: 0.9659 - val_loss: 0.1191
Epoch 7/20
391/391 — 2s 4ms/step - accuracy: 0.9490 - loss: 0.1729 - val_accuracy: 0.9685 - val_loss: 0.1152
Epoch 8/20
391/391 — 2s 4ms/step - accuracy: 0.9534 - loss: 0.1585 - val_accuracy: 0.9686 - val_loss: 0.1104
Epoch 9/20
391/391 — 2s 4ms/step - accuracy: 0.9538 - loss: 0.1534 - val_accuracy: 0.9692 - val_loss: 0.1052
Epoch 10/20
391/391 — 2s 5ms/step - accuracy: 0.9595 - loss: 0.1366 - val_accuracy: 0.9707 - val_loss: 0.1036
Epoch 11/20
391/391 — 3s 7ms/step - accuracy: 0.9582 - loss: 0.1357 - val_accuracy: 0.9716 - val_loss: 0.0993
Epoch 12/20
391/391 — 2s 4ms/step - accuracy: 0.9603 - loss: 0.1280 - val_accuracy: 0.9725 - val_loss: 0.1019
Epoch 13/20
391/391 — 2s 4ms/step - accuracy: 0.9638 - loss: 0.1189 - val_accuracy: 0.9727 - val_loss: 0.0979
Epoch 14/20
391/391 — 2s 4ms/step - accuracy: 0.9654 - loss: 0.1142 - val_accuracy: 0.9729 - val_loss: 0.0986
Epoch 15/20
391/391 — 2s 4ms/step - accuracy: 0.9657 - loss: 0.1065 - val_accuracy: 0.9720 - val_loss: 0.0980
Epoch 16/20
391/391 — 2s 4ms/step - accuracy: 0.9665 - loss: 0.1096 - val_accuracy: 0.9721 - val_loss: 0.1017
```

```
[8]
✓ 1s
# 1. Print final metrics
train_acc = history.history['accuracy'][-1]
val_acc = history.history['val_accuracy'][-1]
train_loss = history.history['loss'][-1]
val_loss = history.history['val_loss'][-1]

print(f"Final Training Accuracy: {train_acc*100:.2f}%")
print(f"Final Validation Accuracy: {val_acc*100:.2f}%")
print(f"Final Training Loss: {train_loss:.4f}")
print(f"Final Validation Loss: {val_loss:.4f}")

# 2. Plot Training vs Validation Accuracy
plt.figure(figsize=(12, 4))

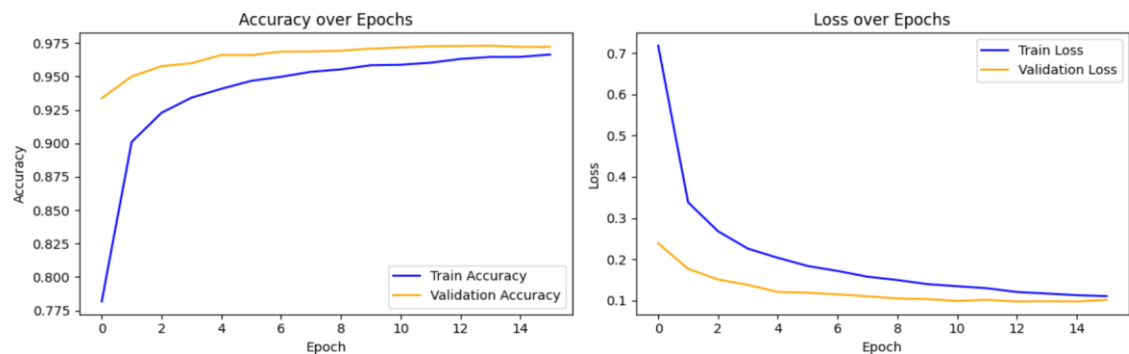
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', color='blue')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', color='orange')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# 3. Plot Training vs Validation Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', color='blue')
plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

# 4. Analyze for Overfitting
accuracy_gap = train_acc - val_acc
if accuracy_gap > 0.05:
    print(f"⚠ Warning: Potential Overfitting detected. Gap: {accuracy_gap*100:.2f}%")
elif val_loss > train_loss * 1.5:
    print("⚠ Warning: Validation loss is significantly higher than training loss (Overfitting sign).")
else:
    print("✅ Model is well-generalized: Training and Validation metrics are closely aligned.")
```

```
Final Training Accuracy: 96.63%
Final Validation Accuracy: 97.21%
Final Training Loss: 0.1108
Final Validation Loss: 0.1017
```



✅ Model is well-generalized: Training and Validation metrics are closely aligned.

```
[9]
✓ ts
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Make Predictions ---
predictions = model.predict(test_images)
predicted_classes = np.argmax(predictions, axis=1)

# Check if test_labels is one-hot or integer
if len(test_labels.shape) > 1 and test_labels.shape[1] > 1:
    true_classes = np.argmax(test_labels, axis=1) # It was one-hot
else:
    true_classes = test_labels # It was already integers

# --- 2. Visualize the Results ---
def visualize_predictions(start_index, num_images=10):
    plt.figure(figsize=(15, 6))
    for i in range(num_images):
        index = start_index + i
        if index >= len(test_images): break

        plt.subplot(2, 5, i + 1)
        # Reshape back to 28x28
        img = test_images[index].reshape(28, 28)

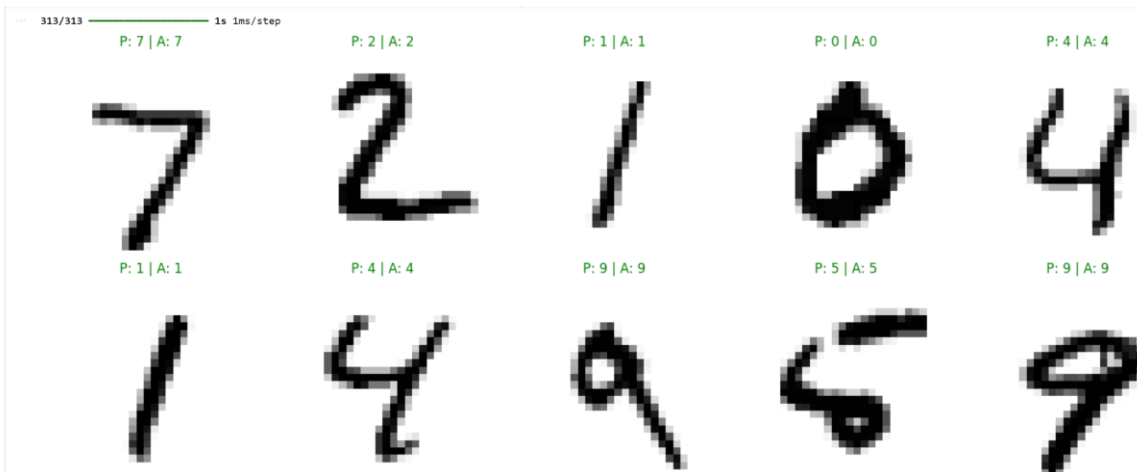
        predicted_label = predicted_classes[index]
        actual_label = int(true_classes[index]) # Ensure it's an int for display

        color = 'green' if predicted_label == actual_label else 'red'

        plt.imshow(img, cmap='binary')
        plt.title(f"P: {predicted_label} | A: {actual_label}", color=color)
        plt.axis('off')

    plt.tight_layout()
    plt.show()

visualize_predictions(start_index=0, num_images=10)
```



SAVING THE MODEL USING PICKLE:

```
import pickle

model_to_save = {'config': model.get_config(), 'weights': model.get_weights()}

with open('mnist_model.pkl', 'wb') as f:
    pickle.dump(model_to_save, f)

print("✓ Model saved as mnist_model.pkl")
```

... ✓ Model saved as mnist_model.pkl

LOADING THE MODEL USING PICKLE:

```
with open('mnist_model.pkl', 'rb') as f:
    data = pickle.load(f)

loaded_model = models.Sequential.from_config(data['config'])
loaded_model.set_weights(data['weights'])

loaded_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

print("✅ Model loaded from directory")
```

... ✅ Model loaded from directory

TESTING:

```
[13]
26 import matplotlib.pyplot as plt
import numpy as np

loss, accuracy = loaded_model.evaluate(test_images, test_labels, verbose=0)
print(f"✅ Loaded Model Test Accuracy: {accuracy * 100:.2f}%")
print("-" * 30)

# 2. Make Predictions for Visualization
num_images = 10
predictions = loaded_model.predict(test_images[:num_images])
predicted_classes = np.argmax(predictions, axis=1)

# 3. Plotting the Grid
plt.figure(figsize=(15, 6))
for i in range(num_images):
    plt.subplot(2, 5, i + 1)

    # Reshape back to 28x28 for display
    img = test_images[i].reshape(28, 28)

    pred = predicted_classes[i]
    actual = test_labels[i]

    if isinstance(actual, np.ndarray):
        actual = np.argmax(actual)

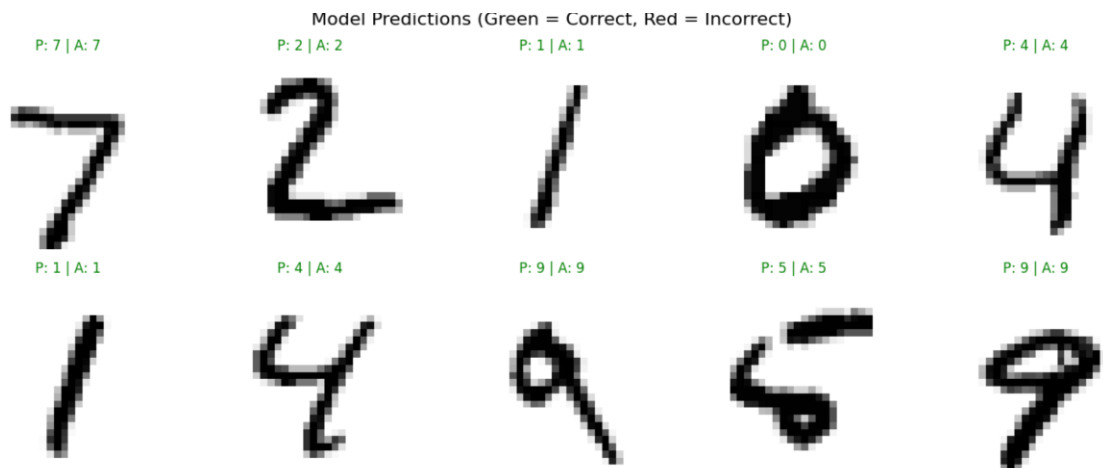
    color = 'green' if pred == actual else 'red'

    plt.imshow(img, cmap='binary')
    plt.title(f"P: {pred} | A: {actual}", color=color)
    plt.axis('off')

plt.suptitle("Model Predictions (Green = Correct, Red = Incorrect)", fontsize=16)
plt.tight_layout()
plt.show()
```

... ✅ Loaded Model Test Accuracy: 97.19%

1/1 0s 94ms/step



6 JARS:

1. The Data Jar

Description: This jar focuses on the collection, cleaning, and preprocessing of information. For this project, the data consists of grayscale images of handwritten digits.

Implementation:

- Loading the MNIST dataset (70,000 images).
- **Normalization:** Scaling pixel values from [0, 255] to [0, 1] using `astype('float32') / 255.0` to help the model converge faster.
- **Reshaping:** Flattening 28 X 28 images into a 784-dimensional vector to suit a Dense (Fully Connected) architecture.

2. The Task Jar

Description: This defines the specific problem the machine learning system is trying to solve.

Implementation:

- **Multi-class Classification:** The goal is to categorize an input image into one of 10 distinct classes (the digits 0 through 9).
- This is a "Supervised Learning" task because we provide the model with labeled examples during training.

3. The Model Jar

Description: The mathematical architecture or "brain" that learns patterns from the data.

Implementation:

- **Sequential Neural Network:** A feed-forward architecture.
- **Layers:** A 64-neuron hidden layer, a 32-neuron hidden layer, and a 10-neuron output layer.
- **Activations:** ReLU for hidden layers to handle non-linearity and Softmax for the output layer to generate probability scores for each digit.

4. The Loss Jar

Description: The objective function that quantifies the "error" of the model's predictions compared to the actual labels.

Implementation:

- **Sparse Categorical CrossEntropy:** This function measures the performance of a classification model whose output is a probability value between 0 and 1.
- **Edge Optimization:** "Sparse" is chosen specifically to handle integer labels directly, which is more memory-efficient for edge computing than one-hot encoded labels.

5. The Learning Jar

Description: The algorithm used to update the model's weights to minimize the loss.

Implementation:

- **Optimizer:** Adam (Adaptive Moment Estimation), which efficiently adjusts the learning rate during training.
- **Training Loop:** model.fit() with a batch size of 128 over 20 epochs.
- **Regularization:** Using Dropout layers and EarlyStopping to prevent overfitting, ensuring the model generalizes well to new data.

6. The Accuracy (Evaluation) Jar

Description: The metrics used to judge if the model is "good enough" for deployment.

Implementation:

- **Quantitative:** The accuracy metric calculated on the unseen Test Dataset.
- **Qualitative:** Visual verification using a grid of predictions, where green labels indicate correct classification and red indicates errors.
- **Edge Readiness:** Verifying the model size (e.g., ~205 KB) to ensure it fits within the memory constraints of a microcontroller.

Task-2: Data Acquisition Workflow via Edge Impulse

The following steps outline the process for collecting and preparing real-world data for edge deployment:

1. **Account Initialization:** Create and verify an account on the Edge Impulse Studio platform. This dashboard serves as the central hub for project management, data storage, and model training.
2. **Project Setup:** Open the Data Acquisition module within the project dashboard. This section is dedicated to capturing the raw inputs (vision, audio, or motion) required for the machine learning pipeline.
3. **Device Integration:** Select the "Connect Data" option to link a data-source device. This allows for real-time data ingestion from hardware such as development boards, PCs, or mobile devices.
4. **Remote Connection:** Use a smartphone to scan the provided QR code. This establishes a secure link between the phone's camera and the Edge Impulse cloud environment.
5. **Data Labeling:** Assign a specific Class Label (e.g., "Fan", "Light"). Accurate labeling is vital as it provides the ground truth for the model's supervised learning process.
6. **Dataset Partitioning:** Once collection is complete, split the images into Training and Testing sets. This ensures the model is trained on one portion of data and validated on another to check for generalization.

EDGE IMPULSE

Dashboard

Devices

Data acquisition

Experiments

EON Tuner

Impulse design

Create impulse

Live classification

Model testing

Post-processing

Deployment

Versioning

GETTING STARTED

Upgrade Plan

Get access to higher job limits and more collaborators.

View plans

dhaya / dhaya-project-1PERSONAL

Target: Raspberry Pi 5

DatasetData sourcesSynthetic dataLabeling queue (0)AI labeling

DATA COLLECTED20 items

TRAIN / TEST SPLIT80% / 20%

Collect data

Connect a device to start building your dataset.

Dataset

Training (16)Test (4)Post-processing (0)

notebook.6eed09ns

notebook

specs.6eect0h2

specs

watch.6eecru5h

watch

mouse.6eecqv5g

mouse

specs.6eecormo

specs

watch.6eecnnd2

watch

mouse.6eecn3tc

mouse

watch.6ebbm1a5

watch

mouse.6ebbhik2

mouse

specs.6eecormo

specs.6eecormo

specs

100%

Metadata

Resume tutorial

EDGE IMPULSE

Dashboard

Devices

Data acquisition

Experiments

EON Tuner

Impulse design

Create impulse

Live classification

Model testing

Post-processing

Deployment

Versioning

GETTING STARTED

Upgrade Plan

Get access to higher job limits and more collaborators.

View plans

dhaya / dhaya-project-1PERSONAL

Target: Raspberry Pi 5

DatasetData sourcesSynthetic dataLabeling queue (0)AI labeling

DATA COLLECTED20 items

TRAIN / TEST SPLIT80% / 20%

Collect data

Connect a device to start building your dataset.

Dataset

Training (16)Test (4)Post-processing (0)

SAMPLE NAME	LABELS	ADDED
notebook.6eed09ns	notebook	Today, 21:40:19
specs.6eect0h2	specs	Today, 21:38:31
watch.6eecru5h	watch	Today, 21:37:56
mouse.6eecqv5g	mouse	Today, 21:37:24
specs.6eecormo	specs	Today, 21:36:15
watch.6eecnnd2	watch	Today, 21:35:38
mouse.6eecn3tc	mouse	Today, 21:34:45
watch.6ebbm1a5	watch	Yesterday, 17:19:31
mouse.6ebbhik2	mouse	Yesterday, 17:17:05
notebook.6ebbg79q	notebook	Today, 21:25:53
notebook.6ebbfuij	notebook	Yesterday, 17:16:11

notebook.6eed09ns

notebook

classmate

100%

Metadata

Resume tutorial

Total pictures are 20 out of which 16 are for training and 4 are for testing to get a 80:20 ratio

